

**ОТЧЕТ**  
по лабораторной работе №2  
по курсу «Логика»  
на тему «Оценка времени выполнения программ»

Выполнили:

студенты группы 25ВВВ4

Волковец А.С

Пантюхин З.С

Приняли:

к.т.н. Юрова О.В.

к.т.н. Деев М.В.

## 1. Цель работы

Познакомится и научиться применять простейшие алгоритмы

## 2. Лабораторное задание

Задание 1:

1. Вычислить порядок сложности программы ( $O$ -символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

## 3. Ход работы

Задание 1.

1. Вычислили порядок сложности программы ( $O$ -символику).

Умножение двух квадратных матриц размером  $N \times N$  содержит 3 вложенных цикла, каждый из которых выполняется  $N$  раз:

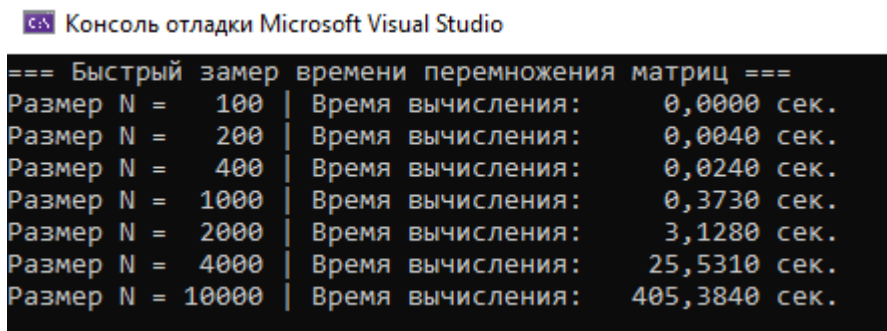
1. Внешний цикл по  $i$  (строки):  $N$  итераций.
2. Средний цикл по  $j$  (столбцы):  $N$  итераций.
3. Внутренний цикл по  $r$  (суммирование произведений элементов):  $N$  итераций.

Общее количество операций умножения и сложения пропорционально  $N \cdot N \cdot N = N^3$ .

Временная сложность алгоритма составляет  $O(N^3)$ .

Для хранения трёх матриц  $A$ ,  $B$  и  $C$  требуется  $O(N^2)$  памяти.

2. Оценили время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.



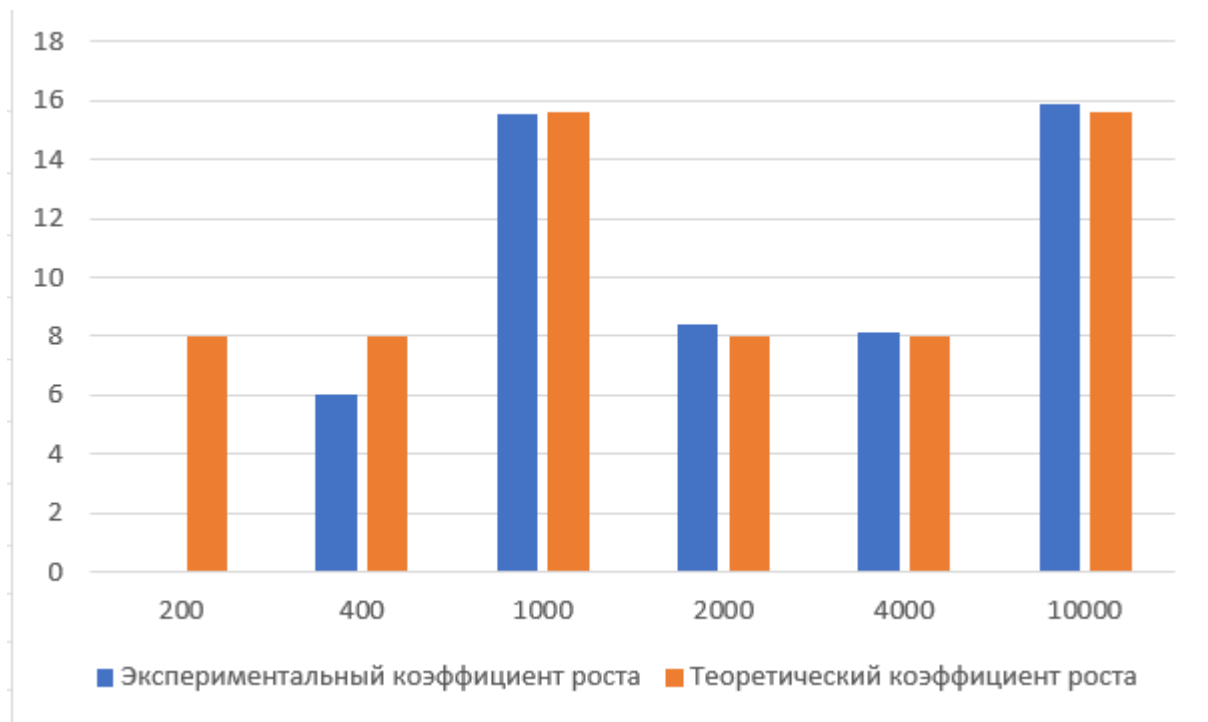
```

Консоль отладки Microsoft Visual Studio

=== Быстрый замер времени перемножения матриц ===
Размер N = 100 | Время вычисления: 0,0000 сек.
Размер N = 200 | Время вычисления: 0,0040 сек.
Размер N = 400 | Время вычисления: 0,0240 сек.
Размер N = 1000 | Время вычисления: 0,3730 сек.
Размер N = 2000 | Время вычисления: 3,1280 сек.
Размер N = 4000 | Время вычисления: 25,5310 сек.
Размер N = 10000 | Время вычисления: 405,3840 сек.
  
```

3. Построили график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

| Размер N | Экспериментальное время (сек) | Экспериментальный коэффициент роста | Теоретический коэффициент роста |
|----------|-------------------------------|-------------------------------------|---------------------------------|
| 100      | 0,0000                        | —                                   | —                               |
| 200      | 0,0040                        | —                                   | 8,00                            |
| 400      | 0,0240                        | 6,00                                | 8,00                            |
| 1000     | 0,3730                        | 15,54                               | 15,63                           |
| 2000     | 3,1280                        | 8,39                                | 8,00                            |
| 4000     | 25,5310                       | 8,16                                | 8,00                            |
| 10000    | 405,3840                      | 15,88                               | 15,63                           |



## Задание 2:

```

=== Набор данных: Случайный массив (N = 50000) ===
1. Сортировка Шелла:      0,1320 сек.
2. Своя QuickSort (qs):   0,0040 сек.
3. Стандартная qsort:    0,0080 сек.

=== Набор данных: Возрастающая последовательность (N = 50000) ===
1. Сортировка Шелла:      0,0000 сек.
2. Своя QuickSort (qs):   0,0010 сек.
3. Стандартная qsort:    0,0030 сек.

=== Набор данных: Убывающая последовательность (N = 50000) ===
1. Сортировка Шелла:      0,2530 сек.
2. Своя QuickSort (qs):   0,0020 сек.
3. Стандартная qsort:    0,0040 сек.

=== Набор данных: 1/2 возрастающая, 1/2 убывающая (N = 50000) ===
1. Сортировка Шелла:      0,1400 сек.
2. Своя QuickSort (qs):   0,5950 сек.
3. Стандартная qsort:    0,0130 сек.

```

Для случайного массива самой эффективной оказалась своя реализация quicksort

Для Возрастающей последовательности – Сортировка Шелла

Для Убывающей – своя реализация QuickSort

Для 50% возрастающая, 50% убывающая – Стандартная Quicksort

```

=== Набор данных: Случайный массив (N = 100000) ===
1. Сортировка Шелла:      0,4290 сек.
2. Своя QuickSort (qs):   0,0180 сек.
3. Стандартная qsort:    0,0410 сек.

=== Набор данных: Возрастающая последовательность (N = 100000) ===
1. Сортировка Шелла:      0,0000 сек.
2. Своя QuickSort (qs):   0,0010 сек.
3. Стандартная qsort:    0,0310 сек.

=== Набор данных: Убывающая последовательность (N = 100000) ===
1. Сортировка Шелла:      0,7200 сек.
2. Своя QuickSort (qs):   0,0020 сек.
3. Стандартная qsort:    0,0150 сек.

=== Набор данных: 1/2 возрастающая, 1/2 убывающая (N = 100000) ===
1. Сортировка Шелла:      0,3140 сек.
2. Своя QuickSort (qs):   1,4060 сек.
3. Стандартная qsort:    0,0560 сек.

```

После увеличения набора данных вдвое ситуация не изменилась

#### 4. Листинг

##### Файл Task1.cpp

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#include <locale>
```

```
void measure_matrix_multiply(int N) {
```

```
    int** a = (int**)malloc(N * sizeof(int*));
```

```
    int** b = (int**)malloc(N * sizeof(int*));
```

```
    int** c = (int**)malloc(N * sizeof(int*));
```

```
    for (int i = 0; i < N; i++) {
```

```
        a[i] = (int*)malloc(N * sizeof(int));
```

```
        b[i] = (int*)malloc(N * sizeof(int));
```

```
c[i] = (int*)malloc(N * sizeof(int));  
}
```

```
srand((unsigned int)time(NULL));
```

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        a[i][j] = rand() % 100 + 1;  
        b[i][j] = rand() % 100 + 1;  
    }  
}
```

```
clock_t start = clock();
```

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        int elem_c = 0;  
        for (int r = 0; r < N; r++) {  
            elem_c += a[i][r] * b[r][j];  
        }  
        c[i][j] = elem_c;  
    }  
}
```

```
clock_t end = clock();
```

```
double time_spent = (double)(end - start) / CLOCKS_PER_SEC;
```

```
printf("Размер N = %5d | Время вычисления: %10.4f сек.\n", N, time_spent);
```

```
for (int i = 0; i < N; i++) {  
    free(a[i]); free(b[i]); free(c[i]);  
}  
free(a); free(b); free(c);  
}
```

```
int main2(void) {  
    setlocale(LC_ALL, "rus");
```

```
int sizes[] = { 100, 200, 400, 1000, 2000, 4000, 10000};  
int num_sizes = sizeof(sizes) / sizeof(sizes[0]);
```

```
printf("=== Замер времени перемножения матриц ===\n");  
for (int i = 0; i < num_sizes; i++) {  
    measure_matrix_multiply(sizes[i]);  
}
```

```
return 0;  
}
```

## Файл Task2.cpp

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

```
#include <locale.h>
```

```
#define N 50000
```

```
#define N2 100000
```

```
// 1. Сортировка Шелла (из методического указания)
```

```
void shell(int* items, int count) {
```

```
    int i, j, gap, k;
```

```
    int x, a[5];
```

```
    a[0] = 9; a[1] = 5; a[2] = 3; a[3] = 2; a[4] = 1;
```

```
    for (k = 0; k < 5; k++) {
```

```
        gap = a[k];
```

```
        for (i = gap; i < count; ++i) {
```

```
            x = items[i];
```

```
            for (j = i - gap; (j >= 0) && (x < items[j]); j = j - gap) {
```

```
                items[j + gap] = items[j];
```

```
            }
```

```
            items[j + gap] = x;
```

```
        }
```

```
    }
```

```
}
```

// 2. Быстрая сортировка с оптимизацией глубины стека  $O(\log N)$

```
void qs(int* items, int left, int right) {  
    while (left < right) {  
        int i = left, j = right;  
        int x = items[left + (right - left) / 2];  
  
        do {  
            while ((items[i] < x) && (i < right)) i++;  
            while ((x < items[j]) && (j > left)) j--;  
  
            if (i <= j) {  
                int y = items[i];  
                items[i] = items[j];  
                items[j] = y;  
                i++;  
                j--;  
            }  
        } while (i <= j);  
  
        if ((j - left) < (right - i)) {  
            if (left < j) qs(items, left, j);  
            left = i;  
        }  
        else {  
            if (i < right) qs(items, i, right);  
        }  
    }  
}
```

```
        right = j;
    }
}
}
```

```
// Компаратор для qsort
```

```
int compare_ints(const void* a, const void* b) {
    int arg1 = *(const int*)a;
    int arg2 = *(const int*)b;
    if (arg1 < arg2) return -1;
    if (arg1 > arg2) return 1;
    return 0;
}
```

```
// Генерация наборов данных
```

```
void generate_random(int* arr, int n) {
    for (int i = 0; i < n; i++) arr[i] = rand() % 100000;
}
```

```
void generate_ascending(int* arr, int n) {
    for (int i = 0; i < n; i++) arr[i] = i;
}
```

```
void generate_descending(int* arr, int n) {
    for (int i = 0; i < n; i++) arr[i] = n - i;
}
```

```

void generate_half_asc_half_desc(int* arr, int n) {
    int mid = n / 2;
    for (int i = 0; i < mid; i++) arr[i] = i;
    for (int i = mid; i < n; i++) arr[i] = n - i;
}

```

```

void run_test(const char* test_name, void (*gen_func)(int*, int)) {
    int* orig = (int*)malloc(N * sizeof(int));
    int* work = (int*)malloc(N * sizeof(int));

```

```

    if (!orig || !work) {
        printf("Ошибка выделения памяти!\n");
        return;
    }

```

```

    gen_func(orig, N);

```

```

    printf("\n=== Набор данных: %s (N = %d) ===\n", test_name, N);

```

```

    // 1. Тест Сортировки Шелла

```

```

    memcpy(work, orig, N * sizeof(int));

```

```

    clock_t start = clock();

```

```

    shell(work, N);

```

```

    clock_t end = clock();

```

```

    printf("1. Сортировка Шелла:   %10.4f сек.\n", (double)(end - start) /
CLOCKS_PER_SEC);

```

```
// 2. Тест Быстрой сортировки (qs)

memcpy(work, orig, N * sizeof(int));

start = clock();

qs(work, 0, N - 1);

end = clock();

printf("2. Своя QuickSort (qs): %10.4f сек.\n", (double)(end - start) /
CLOCKS_PER_SEC);
```

```
// 3. Тест Стандартной qsort

memcpy(work, orig, N * sizeof(int));

start = clock();

qsort(work, N, sizeof(int), compare_ints);

end = clock();

printf("3. Стандартная qsort: %10.4f сек.\n", (double)(end - start) /
CLOCKS_PER_SEC);
```

```
free(orig);

free(work);

}

void run_test2(const char* test_name, void (*gen_func)(int*, int)) {

    int* orig = (int*)malloc(N2 * sizeof(int));

    int* work = (int*)malloc(N2 * sizeof(int));


    if (!orig || !work) {

        printf("Ошибка выделения памяти!\n");

        return;

    }
```

```
gen_func(orig, N2);

printf("\n=== Набор данных: %s (N = %d) ===\n", test_name, N2);

// 1. Тест Сортировки Шелла
memcpy(work, orig, N2 * sizeof(int));
clock_t start = clock();
shell(work, N2);
clock_t end = clock();

printf("1. Сортировка Шелла:  %10.4f сек.\n", (double)(end - start) /
CLOCKS_PER_SEC);

// 2. Тест Быстрой сортировки (qs)
memcpy(work, orig, N2 * sizeof(int));
start = clock();
qs(work, 0, N2 - 1);
end = clock();

printf("2. Своя QuickSort (qs): %10.4f сек.\n", (double)(end - start) /
CLOCKS_PER_SEC);

// 3. Тест Стандартной qsort
memcpy(work, orig, N2 * sizeof(int));
start = clock();
qsort(work, N2, sizeof(int), compare_ints);
end = clock();

printf("3. Стандартная qsort:  %10.4f сек.\n", (double)(end - start) /
CLOCKS_PER_SEC);
```

```
    free(orig);  
    free(work);  
}
```

```
int main(void) {  
    setlocale(LC_ALL, "rus");  
    setvbuf(stdin, NULL, _IONBF, 0);  
    setvbuf(stdout, NULL, _IONBF, 0);  
    srand((unsigned int)time(NULL));  
  
    run_test("Случайный массив", generate_random);  
    run_test("Возрастающая последовательность", generate_ascending);  
    run_test("Убывающая последовательность", generate_descending);  
    run_test("1/2 возрастающая, 1/2 убывающая", generate_half_asc_half_desc);  
    run_test2("Случайный массив", generate_random);  
    run_test2("Возрастающая последовательность", generate_ascending);  
    run_test2("Убывающая последовательность", generate_descending);  
    run_test2("1/2 возрастающая, 1/2 убывающая", generate_half_asc_half_desc);  
  
    return 0;  
}
```